SC1001: Introduction to Computational Thinking

Week 9: Functions

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

A Quick Aside

I need you to answer questions so I can gauge the pace and content.

- If a question feels too simple, answer it anyway.
- Many quick correct answers tell me we can move faster.
- If most people are comfortable, I will add more advanced material.
- If a question feels too hard, answer the smaller follow-up questions.
- Those smaller questions help us move through the problem step by step.

Overview

1. Read the rules for the 1A2B game
2. Design the game as a sequence of steps
3. Turn those steps into function prototypes
4. Implement each function
5. Combine the functions into a complete program

Bulls & Cows

We will build a Python version of 1A2B, also called Bulls & Cows.

- The program randomly chooses four distinct digits as the secret number.
- In each round, the player guesses four digits.
- A digit earns `1A` if it appears in the secret and is in the same position.
- A digit earns `1B` if it appears in the secret but is in a different position.
- The player wins when a guess earns `4A`.

A Possible Game Transcript

```
Please input 4 digits: 1234
0 A (Bulls) 2 B (Cows)
```

`0A 2B` means two digits are correct, but both are in wrong positions.

Possible secrets include:

`24??` `2?4?` `4?2?` `4??2`

A Possible Game Transcript

```
Please input 4 digits: 1234
0 A (Bulls) 2 B (Cows)
Please input 4 digits: 2478
3 A (Bulls) 0 B (Cows)
```

3A 0B means three positions are correct and the fourth guessed digit is not in the secret.

Possible secrets include:

?478 2?78 24?8 247?

A Possible Game Transcript

```
Please input 4 digits: 1234
0 A (Bulls) 2 B (Cows)
Please input 4 digits: 2478
3 A (Bulls) 0 B (Cows)
Please input 4 digits: 2468
You Win!!
Attempts:  3
```

2468 gives 4A, so the secret is exactly 2468.

The main loop can stop after printing the number of attempts.

Discussion 1: Game Design

To play the 1A2B game, what steps does the program need?

- What should happen before the first guess?
- What should happen during each round?
- What information must the program remember?
- When should the loop stop?

Write the design first as pseudocode or a flowchart.

Discussion 2: Function Interfaces

Based on the algorithm, define a function for each step before implementing the body.

- What does the function do?
- How many parameters does it need?
- What should each parameter represent?
- What should the function return?

Discussion 3: Implement the Functions

Implement each function according to the interface.

- Use `random.shuffle()` to generate the secret.
- Alternative: use `random.sample()` to select digits.
- Check each character against the allowed digit list.
- Return `"1234"` as `["1", "2", "3", "4"]`.
- Compare every guessed digit with every secret digit.

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:

def user_guess(allowed_digits: list[str], count: int) -> list[str]:

def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int
```

Discussion 4: Combine the Functions

Use the functions to complete the game.

- Generate the secret once.
- Repeatedly read and check guesses.
- Count the number of attempts.
- Stop when the result has four bulls.

Dumb Way to Play 1: Try Everyting

- How many possible solutions are there?

Dumb Way to Play 1: Try Everyting

- How many possible solutions are there?

$$10 \cdot 9 \cdot 8 \cot 7 = \frac{10!}{6!} = 5040 \tag{1}$$

Dumb Way to Play 1: Try Everyting

- How many possible solutions are there?

$$10 \cdot 9 \cdot 8 \cot 7 = \frac{10!}{6!} = 5040 \tag{1}$$

- Just try them one by one

Dumb Way to Play 2: Probe

- Guess 1111, 2222, etc.

Dumb Way to Play 2: Probe

- Guess 1111, 2222, etc.
- You'll learn which digits are in the secret

Dumb Way to Play 2: Probe

- Guess 1111, 2222, etc.
- You'll learn which digits are in the secret
- Then try all $4 \cdot 3 \cdot 2 = 24$ orderings

Dumb Ways to Play the Game

Strategy	Worst Case	Average Case
Exhaustive	5040	2520

Dumb Ways to Play the Game

Strategy	Worst Case	Average Case
Exhaustive	5040	2520
Probe	34	17

Discussion 5: Solver Pseudocode

Create pseudocode for an automated solver.

- Generate all possible solutions.
- Make a guess and receive the `A/B` result.
- Remove impossible solutions.
- Generate the next guess

Discussion 6: Solver Code

Write code for the solver pieces.

- Remove excluded solutions after each result.
- Pick a possible solution as the next guess.
- Run the solver many times and measure wall-clock time.

Solver Type Hints

```
from collections.abc import Callable

Guess = tuple[int, ...]
NextGuess = Callable[[set[Guess], set[Guess]], Guess]

def solve(secret: Guess, generate_next_guess: NextGuess) -> int:
```

`NextGuess` names the kind of function the solver can call.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)
```

Start with every possible no-repeat secret, a record of guesses, and a fixed first guess.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)
```

Record the guess, then use `check_guess` to get feedback for the real secret.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)

    if result[0] == WIDTH:
        return steps
```

A result with four bulls means the solver has found the secret.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)

    if result[0] == WIDTH:
        return steps

    for possible_secret in tuple(possible_solutions):
        if check_guess(guess, possible_secret) != result:
            possible_solutions.remove(possible_secret)
```

Loop over a copy of the set so secrets can be removed while checking.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)

    if result[0] == WIDTH:
        return steps

    for possible_secret in tuple(possible_solutions):
        if check_guess(guess, possible_secret) != result:
            possible_solutions.remove(possible_secret)

    guess = generate_next_guess(possible_solutions, guesses)
```

After filtering, delegate the next-guess decision to another function.

A Simple Next Guess

```
def random_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
```

This function receives only the remaining possible solutions.

A Simple Next Guess

```
def random_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
    return random.choice(tuple(solutions))
```

Convert the set to a tuple because `random.choice` needs a sequence.

Discussion 7: Guess Quality

What makes a good guess?

- A good guess should be informative.
- It should exclude many options.
- One baseline scans all 5040 no-repeat guesses.
- Another baseline tries 0000, 1111, and so on.
- Compare average steps and worst case across all secrets.
- More advanced versions use entropy or game-tree search.
- Today: count how many possible outcomes each guess can produce.